UNIT- 1

INTRODUCTION

What is AI :

Since the invention of computers or machines, their capability to perform various tasks went on growing exponentially. Humans have developed the power of computer systems in terms of their diverse working domains, their increasing speed, and reducing size with respect to time. Artificial Intelligence pursues creating the computers or machines as intelligent as human beings.

In today's world, technology is growing very fast, and we are getting in touch with different new technologies day by day.

Here, one of the booming technologies of computer science is Artificial Intelligence which is ready to create a new revolution in the world by making intelligent machines.The Artificial Intelligence is now all around us. It is currently working with a variety of subfields, ranging from general to specific, such as self-driving cars, playing chess, proving theorems, playing music, Painting, etc.

According to the father of Artificial Intelligence, John McCarthy, it is "The science and engineering of making intelligent machines, especially intelligent computer programs" .

Artificial Intelligence is a way of making a computer, a computer-controlled robot, or a software think intelligently, in the similar manner the intelligent humans think.

So, we can define AI as: " It is a branch of computer science by which we can create intelligent machines which can behave like a human, think like humans, and able to make decisions ."

Artificial Intelligence exists when a machine can have human based skills such as learning, reasoning, and solving problems

With Artificial Intelligence you do not need to preprogram a machine to do some work, despite that you can create a machine with programmed algorithms which can work with own intelligence, and that is the awesomeness of AI.

It is believed that AI is not a new technology, and some people says that as per Greek myth, there were Mechanical men in early days which can work and behave like humans.

Why Artificial Intelligence?

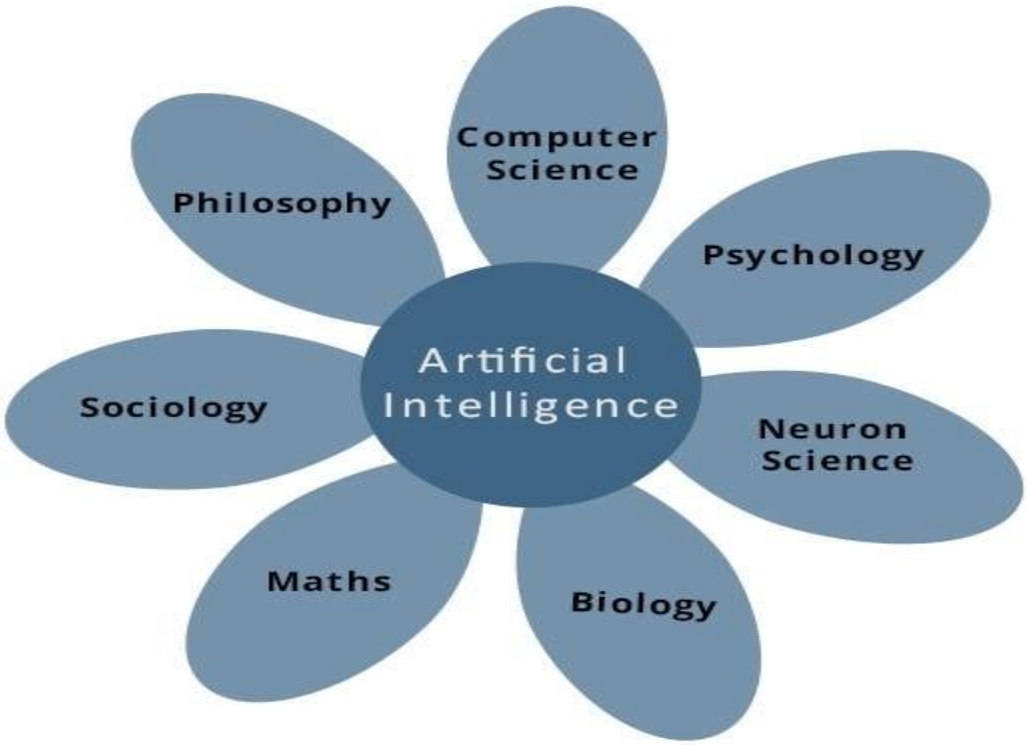
Before Learning about Artificial Intelligence, we should know that what is the importance of AI and why should we learn it. Following are some main reasons to learn about AI:

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk.
- AI opens a path for other new technologies, new devices, and new Opportunities.

Foundation of AI:

Artificial intelligence is a science and technology based on disciplines such as Computer Science, Biology, Psychology, Linguistics, Mathematics, and Engineering. A major thrust of AI is in the development of computer functions associated with human intelligence, such as reasoning, learning, and problem solving.

Out of the following areas, one or multiple areas can contribute to build an intelligent system.



Artificial Intelligence (AI) – Definition and Foundations

Definition of Artificial Intelligence (AI)

Artificial Intelligence (AI) is a branch of computer science that focuses on designing and developing intelligent systems capable of performing tasks that normally require human intelligence, such as learning, reasoning, problem-solving, decision-making, understanding natural language, and recognizing images and speech.

Standard Definitions of AI

John McCarthy (1956):

"Artificial Intelligence is the science and engineering of making intelligent machines, especially intelligent computer programs."

Stuart Russell and Peter Norvig:

"Artificial Intelligence is the study of intelligent agents that perceive their environment and take actions to maximize their chances of achieving goals."

Simple Exam Definition

Artificial Intelligence (AI) is the branch of computer science that enables machines to simulate human intelligence by learning, reasoning, solving problems, making decisions, and understanding language.

Characteristics of AI

1. Learning from data and experience.
2. Reasoning and logical thinking.
3. Problem- solving ability.
4. Decision- making under uncertainty.
5. Knowledge representation.
6. Natural language understanding.
7. Perception through vision and speech.
8. Ability to adapt to changing environments.

Applications of AI

Healthcare (disease diagnosis)

Education (intelligent tutoring systems)

Banking (fraud detection)

E- commerce (recommendation systems)

Robotics

Autonomous vehicles

Virtual assistants

Gaming

Agriculture

Cybersecurity

Foundations of Artificial Intelligence

The **foundations of AI** are the disciplines that contribute concepts, theories, and techniques used to build intelligent systems.

1. Philosophy

Philosophy studies reasoning, knowledge, ethics, and intelligence.

Contributions

- Logical reasoning
- Knowledge representation
- Rational decision- making
- Ethics of AI

Example: Rule- based expert systems.

2. Mathematics

Mathematics provides the formal methods used in AI.

Contributions

- Logic
- Probability
- Statistics
- Linear algebra
- Calculus
- Optimization

Example: Machine learning algorithms use statistics and calculus.

3. Economics

Economics helps AI make rational decisions by maximizing benefits.

Contributions

- Utility theory
- Decision theory
- Cost- benefit analysis
- Game theory

Example: AI selects the best route while minimizing travel time.

4. Neuroscience

Neuroscience studies how the human brain processes information.

Contributions

- Neural networks
- Learning mechanisms
- Memory models

Example: Artificial Neural Networks (ANNs).

5. Psychology

Psychology explains human learning, thinking, memory, and perception.

Contributions

- Learning models
- Cognitive processes
- Problem- solving
- Human behavior

Example: Intelligent tutoring systems.

6. Computer Engineering

Computer engineering provides the hardware and software platforms for AI.

Contributions

- High- speed processors
- GPUs
- Memory systems
- Embedded systems

Example: AI chips for deep learning.

7. Control Theory and Cybernetics

Control theory studies how systems regulate themselves using feedback.

Contributions

- Feedback mechanisms
- Automation
- Robotics

Autonomous systems

Example: Self- driving cars maintain speed and direction automatically.

8. Linguistics

Linguistics helps computers understand and generate human language.

Contributions

Syntax

Semantics

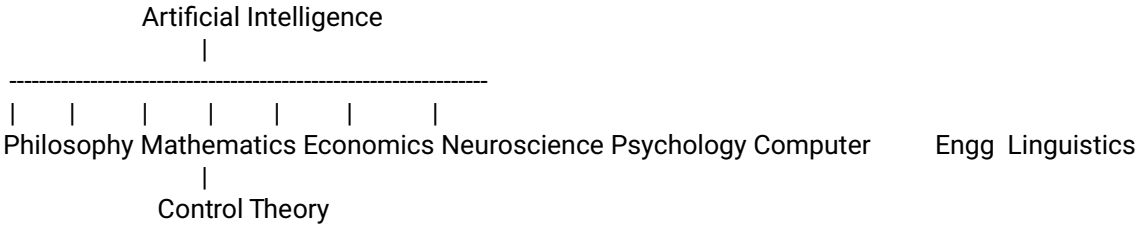
Grammar

Speech recognition

Language translation

Example: Chatbots and virtual assistants.

Diagram: Foundations of AI



Importance of AI Foundations

Provide theoretical knowledge for AI development.

Help build intelligent and rational systems.

Improve machine learning algorithms.

Enable automation and robotics.

Support natural language processing and computer vision.

Enhance decision- making and problem- solving.

Advantages of AI

Reduces human effort.

Improves accuracy.

Operates 24/7.

Handles large amounts of data.

Performs repetitive tasks efficiently.

Supports faster decision- making.

Disadvantages of AI

High development cost. Job displacement in some sectors.

Data privacy concerns.

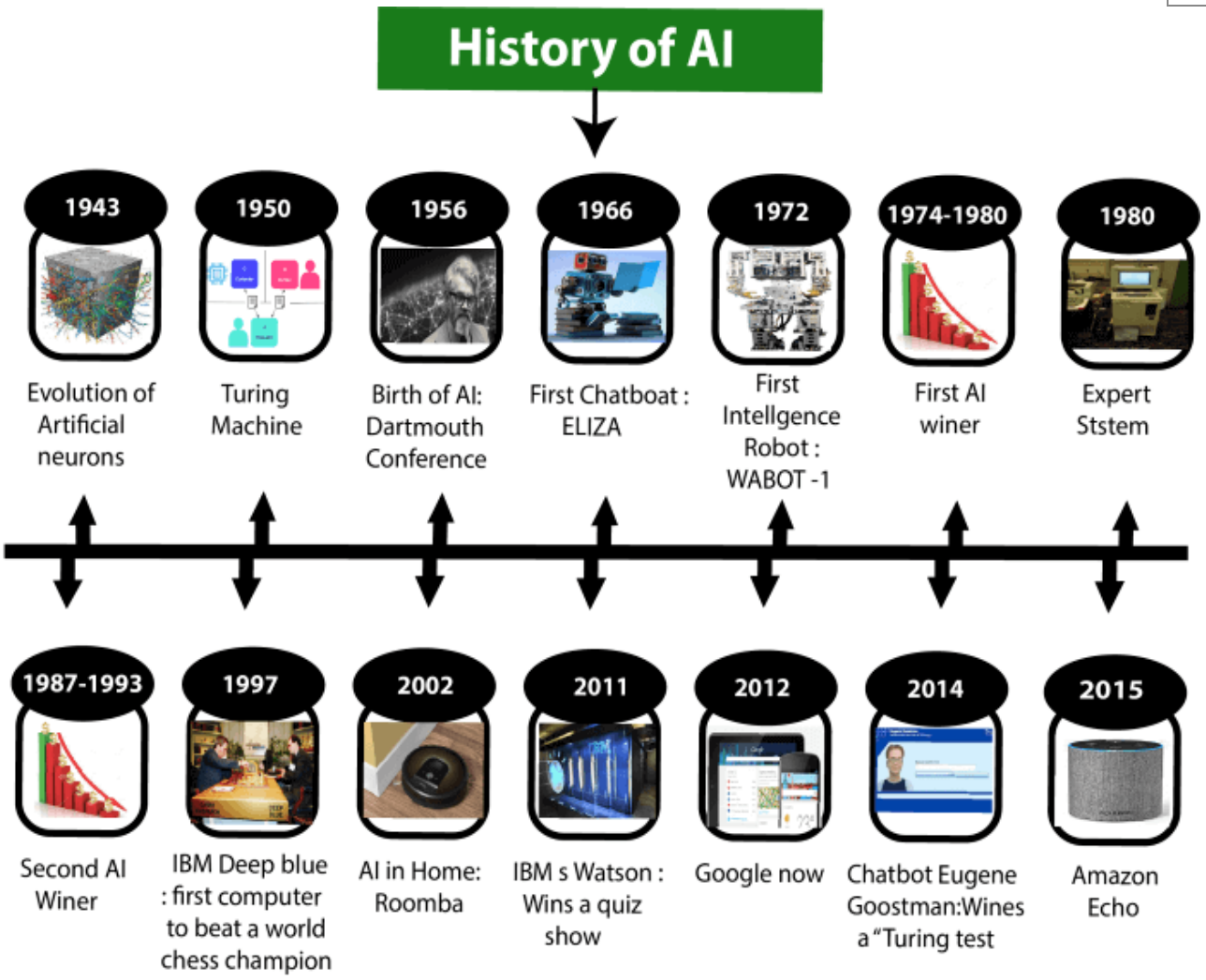
Lack of human emotions and creativity.

Dependence on quality data.

Ethical and security issues.

History of AI :

Artificial Intelligence is not a new word and not a new technology for researchers. This technology is much older than you would imagine. Even there are the myths of Mechanical men in Ancient Greek and Egyptian Myths. Following are some milestones in the history of AI which defines the journey from the AI generation to till date development.



Maturation of Artificial Intelligence (1943- 1952)

- **Year 1943:** The first work which is now recognized as AI was done by Warren McCulloch and Walter pits in 1943. They proposed a model of **artificial neurons** .
- **Year 1949:** Donald Hebb demonstrated an updating rule for modifying the connection strength between neurons. His rule is now called **Hebbian learning** .
- **Year 1950:** The Alan Turing who was an English mathematician and pioneered Machine learning in 1950. Alan Turing publishes **"Computing Machinery and Intelligence"** in which he proposed a test. The test can check the machine's ability to exhibit intelligent behavior equivalent to human intelligence, called a **Turing test** .

The birth of Artificial Intelligence (1952- 1956)

- **Year 1955:** An Allen Newell and Herbert A. Simon created the "first artificial intelligence program"Which was named as **"Logic Theorist"** . This program had proved 38 of 52 Mathematics theorems, and find new and more elegant proofs for some theorems.
- **Year 1956:** The word "Artificial Intelligence" first adopted by American Computer scientist John McCarthy at the Dartmouth Conference. For the first time, AI coined as an academic field.

At that time high-level computer languages such as FORTRAN, LISP, or COBOL were invented. And the enthusiasm for AI was very high at that time.

The golden years- Early enthusiasm (1956- 1974)

- **Year 1966:** The researchers emphasized developing algorithms which can solve mathematical problems. Joseph Weizenbaum created the first chatbot in 1966, which was named as ELIZA.
- **Year 1972:** The first intelligent humanoid robot was built in Japan which was named as WABOT- 1.

The first AI winter (1974- 1980)

- The duration between years 1974 to 1980 was the first AI winter duration. AI winter refers to the time period where computer scientist dealt with a severe shortage of funding from government for AI researches.
- During AI winters, an interest of publicity on artificial intelligence was decreased.

A boom of AI (1980- 1987)

- **Year 1980:** After AI winter duration, AI came back with "Expert System". Expert systems were programmed that emulate the decision-making ability of a human expert.
- In the Year 1980, the first national conference of the American Association of Artificial Intelligence **was held at Stanford University** .

The second AI winter (1987- 1993)

- The duration between the years 1987 to 1993 was the second AI Winter duration.
- Again Investors and government stopped in funding for AI research as due to high cost but not efficient result. The expert system such as XCON was very cost effective.

The emergence of intelligent agents (1993- 2011)

- **Year 1997:** In the year 1997, IBM Deep Blue beats world chess champion, Gary Kasparov, and became the first computer to beat a world chess champion.
- **Year 2002:** for the first time, AI entered the home in the form of Roomba, a vacuum cleaner.
- **Year 2006:** AI came in the Business world till the year 2006. Companies like Facebook, Twitter, and Netflix also started using AI.

Deep learning, big data and artificial general intelligence (2011- present)

- **Year 2011:** In the year 2011, IBM's Watson won jeopardy, a quiz show, where it had to solve the complex questions as well as riddles. Watson had proved that it could understand natural language and can solve tricky questions quickly.
- **Year 2012:** Google has launched an Android app feature "Google now", which was able to provide information to the user as a prediction.
- **Year 2014:** In the year 2014, Chatbot "Eugene Goostman" won a competition in the infamous "Turing test."
- **Year 2018:** The "Project Debater" from IBM debated on complex topics with two master debaters and also performed extremely well.
- Google has demonstrated an AI program "Duplex" which was a virtual assistant and which had taken hairdresser appointment on call, and lady on other side didn't notice that she was talking with the machine.

Now AI has developed to a remarkable level. The concept of Deep learning, big data, and data science are now trending like a boom. Nowadays companies like Google, Facebook, IBM, and Amazon are working with AI and creating amazing devices. The future of Artificial Intelligence is inspiring and will come with high intelligence .

The State of art :

In a world in which humanity exercises total power over machines, super robots must respect three fundamental laws:

1. A robot cannot injure a human being or allow a human being to be harmed by remaining inert;
2. A robot must obey an order given by a human, as long as it does not conflict with the first rule;
3. A robot must protect its existence without conflicting with the other two rules.

We can start with the certainty that AI is not just about robots, as movies often show us. Artificial intelligence is in a search algorithm, in a chatbot, in IoT devices, in cars, and even in autonomous weapons, such as missiles that follow their target.

Today, we can identify two main trends in AI:

Narrow AI (or weak artificial intelligence), which deals with single tasks by integrating only one part of the mind;
General AI (AGI or strong artificial intelligence), which pertains to the ability of a machine to understand or learn any intellectual task that a human being can perform.

The increasingly advanced technology provides researchers with new tools that are capable of achieving important goals, and these tools are great starting points in and of themselves. Among the achievements of recent years, the following are some specific domains:

Machine learning;
Reinforcement learning;
Deep learning;
Natural language processing.

INTELLIGENT AGENTS

Agents and Environments :

An AI system can be defined as the study of the rational agent and its environment. The agents sense the environment through sensors and act on their environment through actuators. An AI agent can have mental properties such as knowledge, belief, intention, etc.

What is an Agent?

An agent can be anything that perceiveits environment through sensors and act upon that environment through actuators. An Agent runs in the cycle of **perceiving** , **thinking** , and **acting** . An agent can be:

- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.
- **Software Agent:** Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

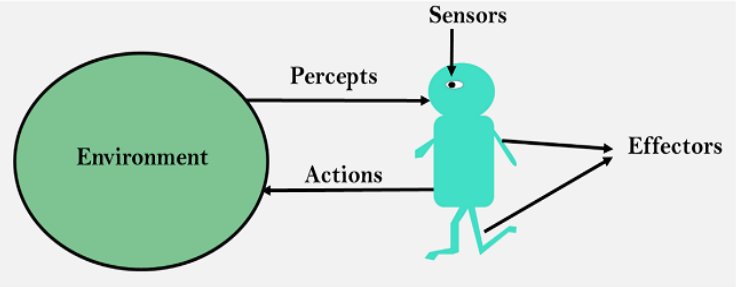
Hence the world around us is full of agents such as thermostat, cellphone, camera, and even we are also agents.

Before moving forward, we should first know about sensors, effectors, and actuators.How tod Nth Highest Salary in SQL

Sensor: Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.

Actuators: Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.

Effectors: Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.



Types of Agents :

Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better action over the time. These are given below:

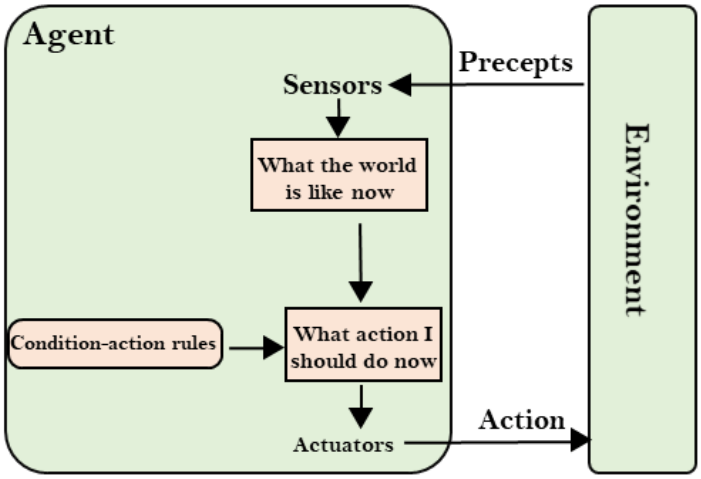
- Simple Reflex Agent
- Model- based reflex agent
- Goal- based agents
- Utility- based agent
- Learning agent

1. Simple Reflex agent:

- The Simple reflex agents are the simplest agents. These agents take decisions on the basis of the current percepts and ignore the rest of the

percept history.

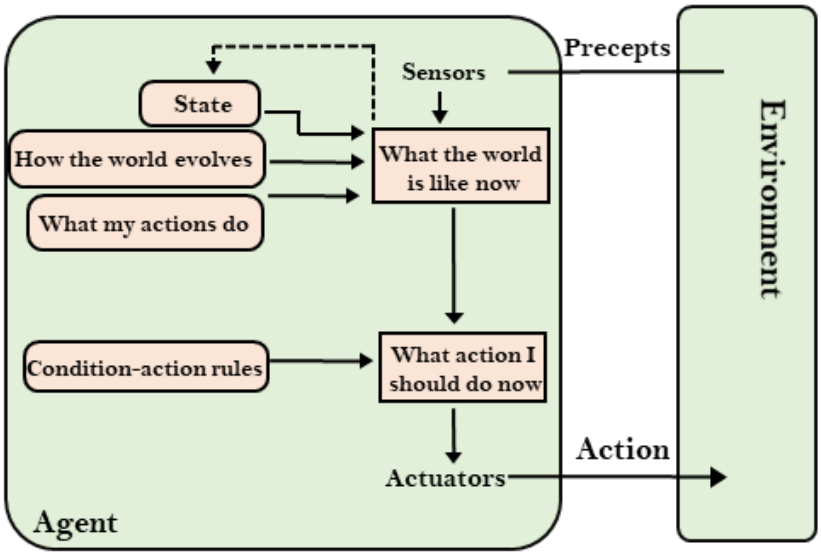
- These agents only succeed in the fully observable environment.
- The Simple reflex agent does not consider any part of percepts history during their decision and action process.
- The Simple reflex agent works on Condition-action rule, which means it maps the current state to action. Such as a Room Cleaner agent, it works only if there is dirt in the room.
- Problems for the simple reflex agent design approach:
  - They have very limited intelligence
  - They do not have knowledge of non-perceptual parts of the current state
  - Mostly too big to generate and to store.
  - Not adaptive to changes in the environment.



2. Model- based reflex agent

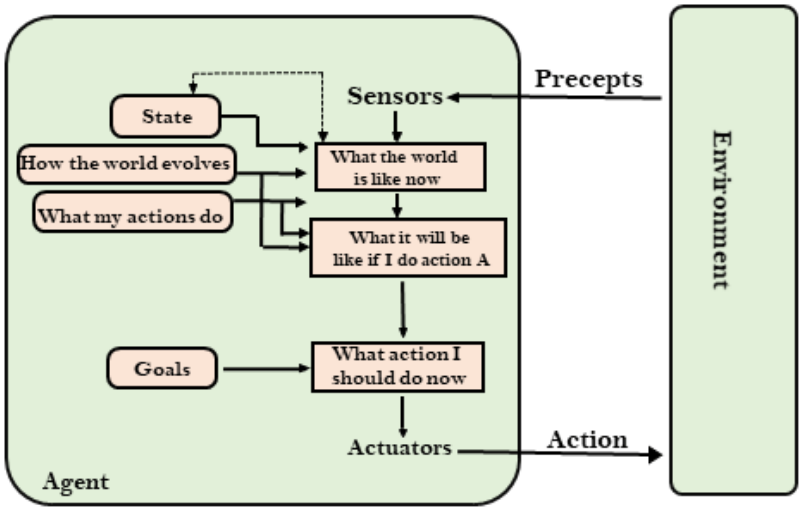
- The Model-based agent can work in a partially observable environment, and track the situation.
- A model- based agent has two important factors:
  - **Model:** It is knowledge about "how things happen in the world," so it is called a Model- based agent.
  - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:

a. How the world evolves
b. How the agent's action affects the world.



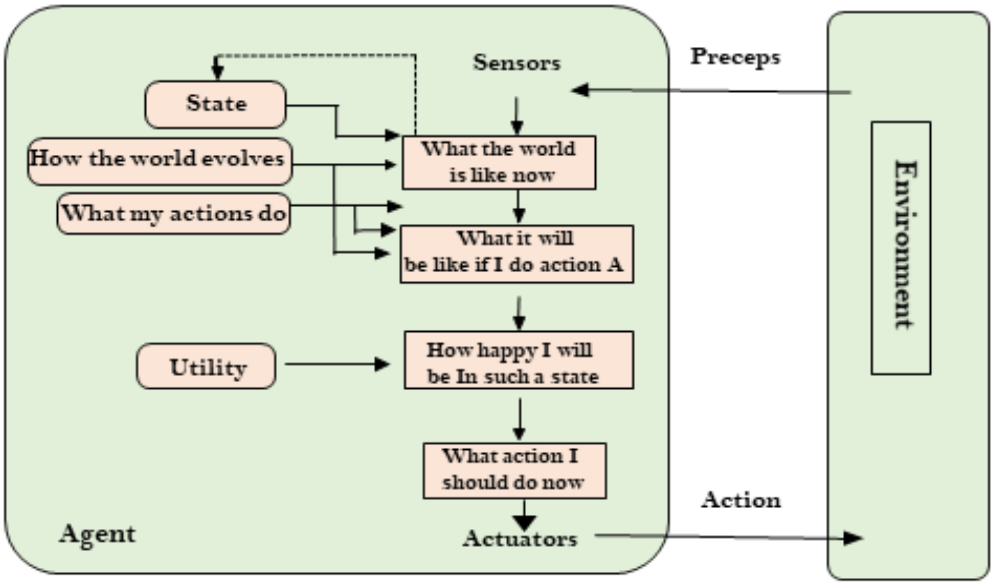
3. Goal- based agents

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its goal which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenario are called searching and planning, which makes an agent proactive.



4. Utility- based agents

- These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agent act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.



5. Learning Agents

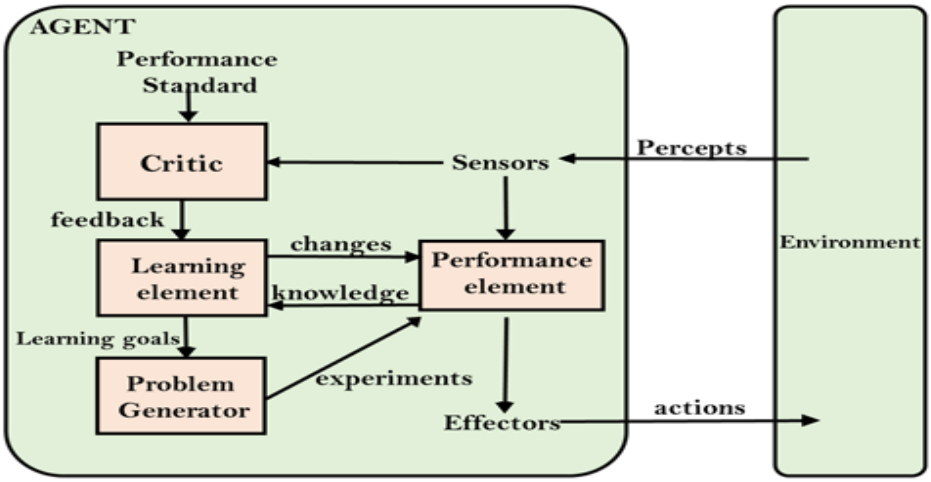
- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:

  a. **Learning element:** It is responsible for making improvements by learning from environment

  b. **Critic:** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.

  c. **Performance element:** It is responsible for selecting external action

  d. **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents are able to learn, analyze performance, and look for new ways to improve the performance.



Intelligent Agents:

An intelligent agent is an autonomous entity which act upon an environment using sensors and actuators for achieving goals. An intelligent agent may learn from the environment to achieve their goals. A thermostat is an example of an intelligent agent.

Following are the main four rules for an AI agent:

- **Rule 1:** An AI agent must have the ability to perceive the environment.
- **Rule 2:** The observation must be used to make decisions.
- **Rule 3:** Decision should result in an action.
- **Rule 4:** The action taken by an AI agent must be a rational action.

Concept Of Rationality:

The rationality of an agent is measured by its performance measure. Rationality can be judged on the basis of following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.
- The sequence of percepts.

Rational Agent:

- A rational agent is an agent which has clear preference, models uncertainty, and acts in a way to maximize its performance measure with all possible actions.
- A rational agent is said to perform the right things. AI is about creating rational agents to use for game theory and decision theory for various real-world scenarios.
- For an AI agent, the rational action is most important because in AI reinforcement learning algorithm, for each best possible action, agent gets the positive reward and for each wrong action, an agent gets a negative reward.

A rational agent always performs right action, where the right action means the action that causes the agent to be most successful in the given percept sequence. The problem the agent solves is characterized by Performance Measure, Environment, Actuators, and Sensors (PEAS).

- Note: Rational agents in AI are very similar to intelligent agents.

The Nature of Environment:

An environment is everything in the world which surrounds the agent, but it is not a part of an agent itself. An environment can be described as a situation in which an agent is present.The environment is where agent lives, operate and provide the agent with something to sense and act upon it. An environment is mostly said to be non- feministic.

Features of Environment

As per Russell and Norvig, an environment can have various features from the point of view of an agent:

1. Fully observable vs Partially Observable
2. Static vs Dynamic
3. Discrete vs Continuous
4. Deterministic vs Stochastic
5. Single- agent vs Multi- agent
6. Episodic vs sequential
7. Known vs Unknown
8. Accessible vs Inaccessible

1. Fully observable vs Partially Observable:

- If an agent sensor can sense or access the complete state of an environment at each point of time then it is **a fully observable** environment, else it is **partially observable** .
- A fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world.
- An agent with no sensors in all environments then such an environment is called as **unobservable** .

2. Deterministic vs Stochastic:

- If an agent's current state and selected action can completely determine the next state of the environment, then such environment is called a deterministic environment.
- A stochastic environment is random in nature and cannot be determined completely by an agent.
- In a deterministic, fully observable environment, agent does not need to worry about uncertainty.

3. Episodic vs Sequential:

- In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
- However, in Sequential environment, an agent requires memory of past actions to determine the next best actions.

4. Single- agent vs Multi- agent

- If only one agent is involved in an environment, and operating by itself then such an environment is called single agent environment.
- However, if multiple agents are operating in an environment, then such an environment is called a multi- agent environment.
- The agent design problems in the multi-agent environment are different from single agent environment.

5. Static vs Dynamic:

- If the environment can change itself while an agent is deliberating then such environment is called a dynamic environment else it is called a static environment.

- Static environments are easy to deal because an agent does not need to continue looking at the world while deciding for an action.
- However for dynamic environment, agents need to keep looking at the world at each action.
- Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

6. Discrete vs Continuous:

- If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.
- A chess gamecomes under discrete environment as there is a finite number of moves that can be performed.
- A self- driving car is an example of a continuous environment.

7. Known vs Unknown

- Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
- In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works in order to perform an action.
- It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.

8. Accessible vs Inaccessible

- If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
- An empty room whose state can be defined by its temperature is an example of an accessible environment.
- Information about an event on earth is an example of Inaccessible environment.

The Structure of Agents :

The task of AI is to design an agent program which implements the agent

function. The structure of an intelligent agent is a combination of architecture and agent program. It can be viewed as:

1. Agent = Architecture + Agent program

Following are the main three terms involved in the structure of an AI agent:

Architecture: Architecture is machinery that an AI agent executes on.

Agent Function: Agent function is used to map a percept to an action.

1. $f:P^* \rightarrow A$

Agent program: Agent program is an implementation of agent function. An agent program executes on the physical architecture to produce function f.

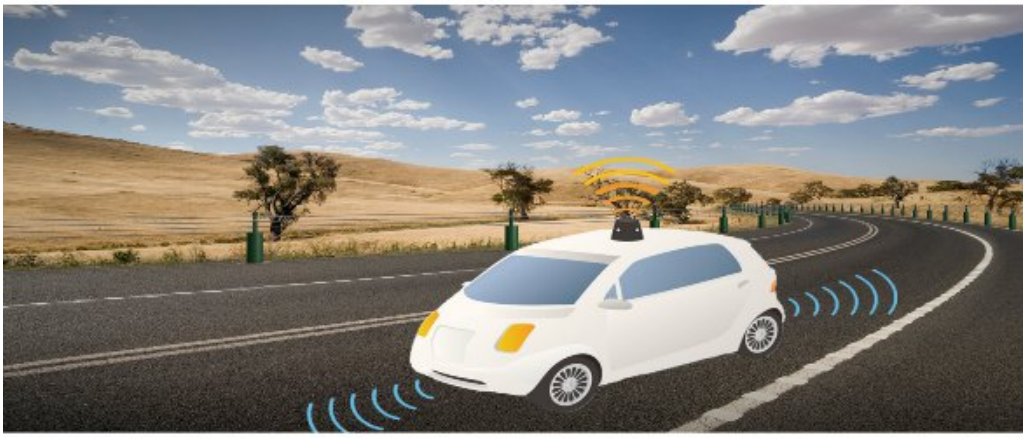
PEAS Representation

PEAS is a type of model on which an AI agent works upon. When we define an AI agent or rational agent, then we can group its properties under PEAS representation model. It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

Here performance measure is the objective for the success of an agent's behavior.

PEAS for self- driving cars:



Let's suppose a self- driving car then PEAS representation will be:

Performance: Safety, time, legal drive, comfort

Environment: Roads, other vehicles, road signs, pedestrian

Actuators: Steering, accelerator, brake, signal, horn

Sensors: Camera, GPS, speedometer, odometer, accelerometer, sonar.

Agent	Performance measure	Environment	Actuators	Sensors
1. Medical Diagnose	o Healthy patient o Minimized cost	o Patient o Hospital o Staff	o Tests o Treatments	Keyboard (Entry of symptoms)

2. Vacuum Cleaner	o Cleanness o Efficiency o Battery life o Security	o Room o Table o Wood floor o Carpet o Various obstacles	o Wheels o Brushes o Vacuum Extractor	o Camera o Dirt detection sensor o Cliff sensor o Bump Sensor o Infrared Wall Sensor
3. Part-picking Robot	o Percentage of parts in correct bins.	o Conveyor belt with parts, o Bins	o Jointed Arms o Hand	o Camera o Joint angle sensors.

Example of Agents with their PEAS representation

UNIT- 2

Solving Problems By Searching

Problem- solving agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

The problem-solving agent perfoms precisely by defining problems and its several solutions.
According to psychology, " a problem-solving refers to a state where we wish to reach to a definite goal from a present state or condition."

Therefore, a problem-solving agent is a **goal-driven agent** and focuses on satisfying the goal.

Steps performed by Problem- solving agent

Goal Formulation: It is the first and simplest step in problem-solving. It organizes the steps/sequence required to formulate one goal out of multiple goals as well as actions to achieve that goal. Goal formulation is based on the current situation and the agent' s performance measure (discussed below).

Problem Formulation: It is the most important step of problem-solving which decides what actions should be taken to achieve the formulated goal. There are following five components involved in problem formulation:

Initial State: It is the starting state or initial step of the agent towards its goal.

Actions: It is the description of the possible actions available to the agent.

Transition Model: It describes what each action does.

Goal Test: It determines if the given state is a goal state.

Path cost: It assigns a numeric cost to each path that follows the goal. The problem-solving agent selects a cost function, which reflects its performance measure. Remember, **an optimal solution has the lowest path cost among all the solutions.**

Note: Initial state, actions , and **transition model** together define the **state-space** of the problem implicitly. State-space of a problem is a set of all states which can be reached from the initial state followed by any sequence of actions.

The state-space forms a directed map or graph where nodes are the states, links between the nodes are actions, and the path is a sequence of states connected by the sequence of actions.

Search: It identifies all the best possible sequence of actions to reach the goal state from the current state. It takes a problem as an input and returns solution as its output.

Solution: It finds the best algorithm out of various algorithms, which may be proven as the best optimal solution.

Execution: It executes the best optimal solution from the searching algorithms to reach the goal state from the current state.

Properties of Search Algorithms:

Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.

Optimality: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.

Time Complexity: Time complexity is a measure of time for an algorithm to complete its task.

Space Complexity: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Example Problems

Basically, there are two types of problem approaches:

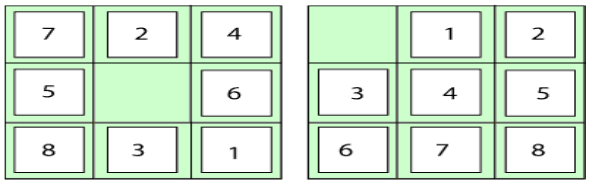
Toy Problem: It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.

Real-world Problem: It is real-world based problems which require solutions. Unlike a toy problem, it does not depend on descriptions, but we can have a general formulation of the problem.

Some Toy Problems

8 Puzzle Problem: Here, we have a 3×3 matrix with movable tiles numbered from 1 to 8 with a blank space. The tile adjacent to the blank space can slide into that space. The objective is to reach a specified goal state similar to the goal state, as shown in the below figure.

In the figure, our task is to convert the current state into goal state by sliding digits into the blank space.



In the above figure, our task is to convert the current(Start) state into goal state by sliding digits into the blank space.

The problem formulation is as follows:

States: It describes the location of each numbered tiles and the blank tile.

Initial State: We can start from any state as the initial state.

Actions: Here, actions of the blank space is defined, i.e., either **left, right, up or down**

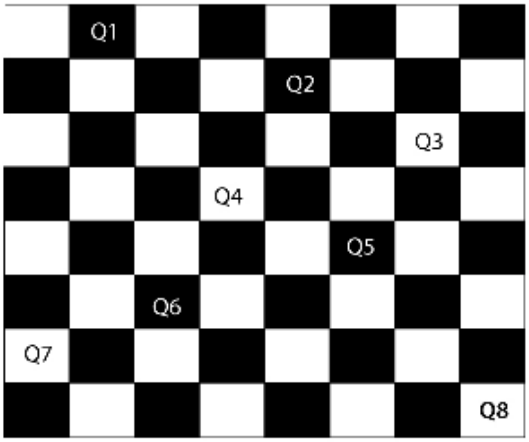
Transition Model: It returns the resulting state as per the given state and actions.

Goal test: It identifies whether we have reached the correct goal- state.

Path cost: The path cost is the number of steps in the path where the cost of each step is **8- queens**

problem: The aim of this problem is to place eight queens on a chessboard in an order where no queen may attack another. A queen can attack other queens either **diagonally or in same row and column.**

From the following figure, we can understand the problem as well as its correct solution.



It is noticed from the above figure that each queen is set into the chessboard in a position where no other queen is placed diagonally, in same row or column. Therefore, it is one right approach to the 8- queens problem.

For this problem, there are two main kinds of formulation:

Incremental formulation: It starts from an empty state where the operator augments a queen at each step.

Following steps are involved in this formulation:

States: Arrangement of any 0 to 8 queens on the chessboard.

Initial State: An empty chessboard

Actions: Add a queen to any empty box.

Transition model: Returns the chessboard with the queen added in a box.

Goal test: Checks whether 8-queens are placed on the chessboard without any attack.

Path cost: There is no need for path cost because only final states are counted.

In this formulation, there is approximately **1.8×10^{14}** possible sequence to investigate.

Complete-state formulation: It starts with all the 8-queens on the chessboard and moves them around, saving from the attacks.

Following steps are involved in this formulation

States: Arrangement of all the 8 queens one per column with no queen attacking the other queen.

Actions: Move the queen at the location where it is safe from the attacks.

This formulation is better than the incremental formulation as it reduces the state space from **1.8×10^{14}** to **2057** , and it is easy to find the solutions.

Some Real- world problems

Traveling salesperson problem(TSP): It is a **touring problem** where the salesman can visit each city only once. The objective is to find the shortest tour and sell- out the stuff in each city.

VLSI Layout problem: In this problem, millions of components and

connections are positioned on a chip in order to minimize the area, circuit-delays, stray- capacitances, and maximizing the manufacturing yield.

The layout problem is split into two parts:

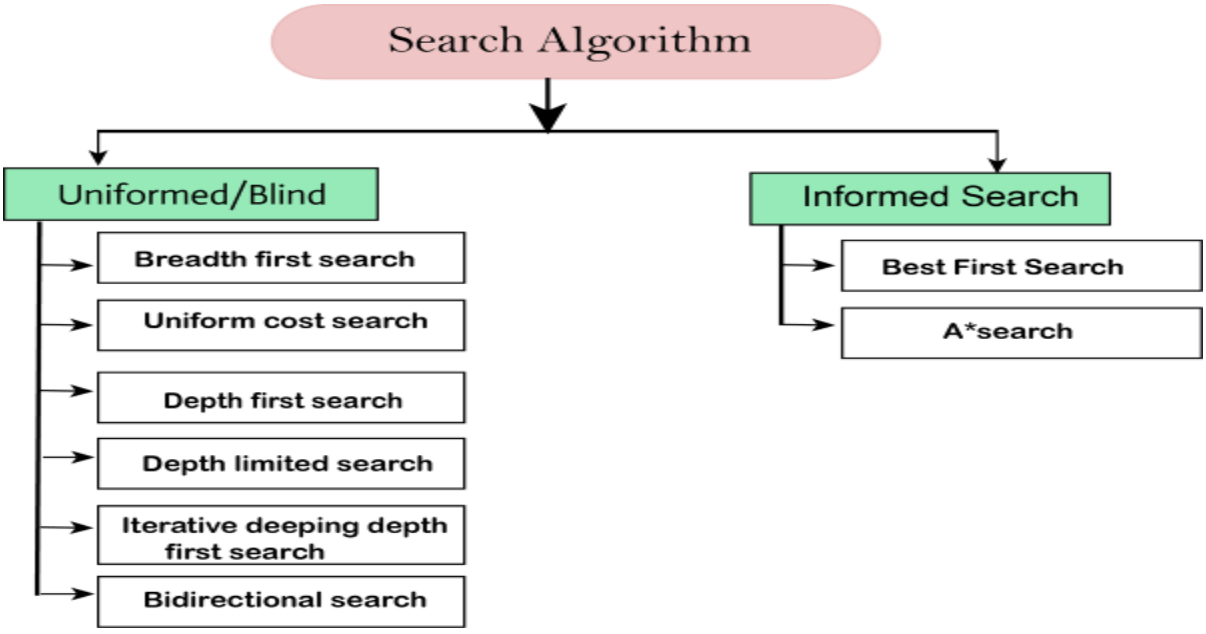
Cell layout: Here, the primitive components of the circuit are grouped into cells, each performing its specific function. Each cell has a fixed shape and size. The task is to place the cells on the chip without overlapping each other.

Channel routing: It finds a specific route for each wire through the gaps between the cells.

Protein Design: The objective is to find a sequence of amino acids which will fold into 3D protein having a property to cure some disease.

Searching for Solution

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search.It examines each node of the tree until

it achieves the goal node.

It can be divided into five main types:

- Breadth- first search
- Uniform cost search
- Depth- first search
- Iterative deepening depth- first search
- Bidirectional Search

1. Breadth- first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth- first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth- first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

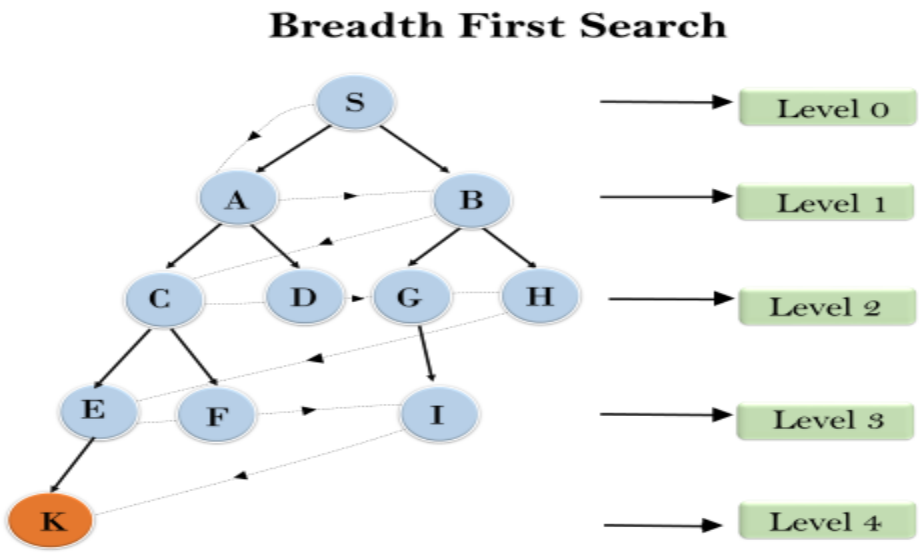
Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S- - - > A- - - >B- - - - >C- - - >D- - - - >G- - - >H- - - >E- - - - >F- - - - >I- - - - >K



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$T(b) = 1+b^2+b^3+.....+b^d = O(b^d)$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth- first Search

- Depth-first search isa recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Note: Backtracking is an algorithm technique for finding all possible solutions using recursion.

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

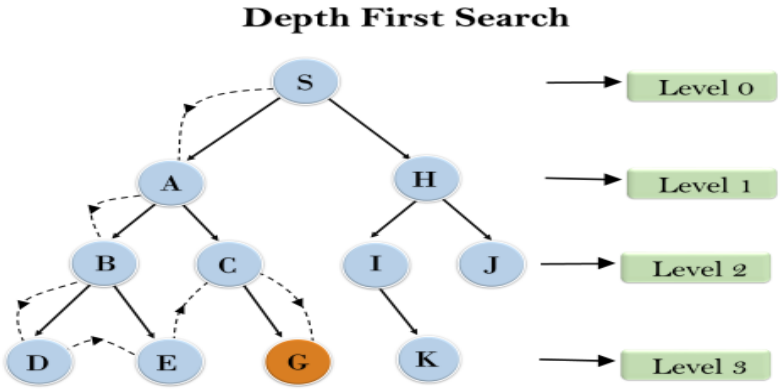
- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

Root node- - - >Left node - - - - > right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node

traversed by the algorithm. It is given by:

$T(n)= 1+ n^2+ n^3 +.....+ n^m=O(n^m)$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)** .

Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

3. Depth- Limited Search Algorithm:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth- limited search can be terminated with two Conditions of failure:

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

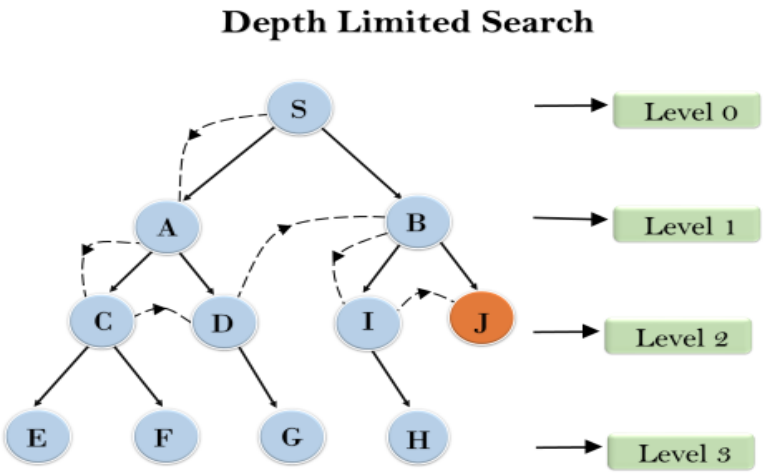
Advantages:

Depth- limited search is Memory efficient.

Disadvantages:

- Depth- limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

Example:



Completeness: DLS search algorithm is complete if the solution is above the depth- limit.

Time Complexity: Time complexity of DLS algorithm is **O(b $^{\ell}$)**.

Space Complexity: Space complexity of DLS algorithm is O **(b×ℓ)** .

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if ℓ>d.

4. Uniform- cost Search Algorithm:

Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph. This algorithm comes into play when a different cost is available for each edge. The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost. Uniform-cost search expands nodes according to their path costs form the root node. It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue. It gives maximum priority to the lowest cumulative cost. Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

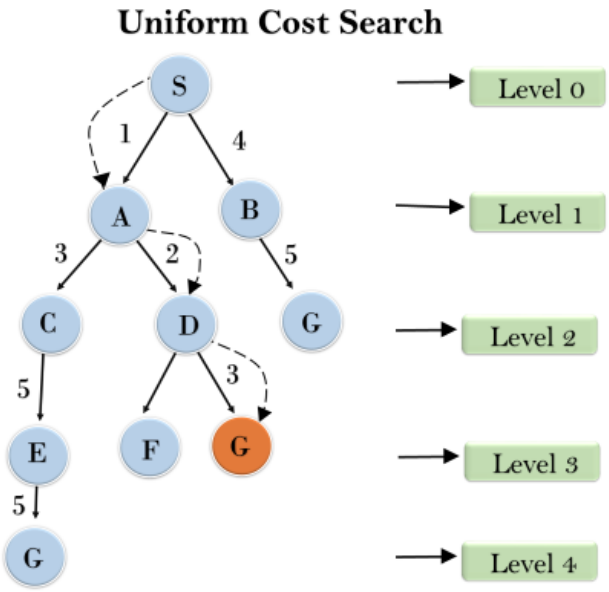
Advantages:

- Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

- It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

Example:



Completeness:

Uniform- cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity:

Let C* **is Cost of the optimal solution** , and **ε** is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε.

Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1+[C^*/\varepsilon]})$.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1+[C^*/\varepsilon]})$.

Optimal:

Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

5. Iterative deepeningdepth - first Search:

The iterative deepening algorithm is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast search and depth- first search's memory efficiency.

The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Advantages:

- Itcombines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

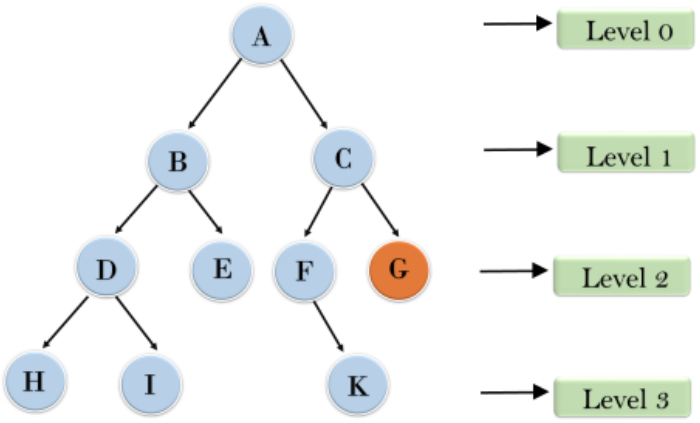
Disadvantages:

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

Example:

Following tree structure is showing the iterative deepening depth-first search. IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:

Iterative deepening depth first search



1'st Iteration-----> A
2'nd Iteration----> A, B, C
3'rd Iteration------>A, B, D, E, C, F, G
4'th Iteration------>A, B, D, H, I, E, C, F, K, G
In the fourth iteration, the algorithm will find the goal node.

Completeness:

This algorithm is complete is ifthe branching factor is finite.

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexity is **O(b ^{d})**.

Space Complexity:

The space complexity of IDDFS will be **O(bd)** .

Optimal:

IDDFS algorithm is optimal if path cost is a non- decreasing function of the depth of the node.

6. Bidirectional Search Algorithm:

Bidirectional search algorithm runs two simultaneous searches, one form initial state called as forward-search and other from goal node called as backward-

search, to find the goal node. Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

- Bidirectional search is fast.
- Bidirectional search requires less memory

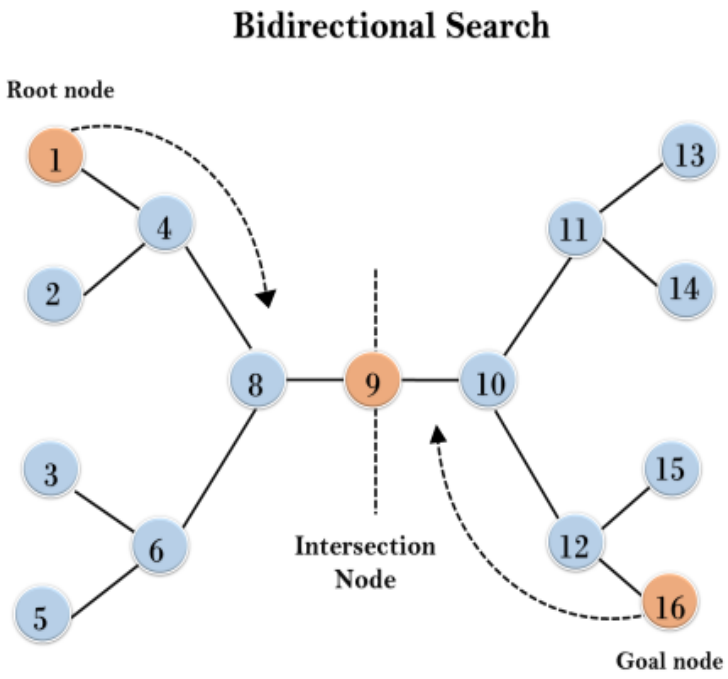
Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- **In bidirectional search, one should know the goal state in advance.**

Example:

In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.



Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: Time complexity of bidirectional search using BFS is **$O(b^d)$**.

Space Complexity: Space complexity of bidirectional search is **$O(b^d)$**.

Optimal: Bidirectional search is Optimal.

Informed Search Strategy:

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

Informed search can solve much complex problem which could not be solved in another way.

Heuristics function: Heuristic is a function which is used in Informed Search, and

it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by h(n), and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Admissibility of the heuristic function is given as

1. h(n) <= h*(n)

Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues unit a goal state is found.

An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search
2. A* Search

1.) Best- first Search Algorithm (Greedy Search):

Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is

estimated by heuristic function, i.e.

1. f(n)= g(n).

Were, h(n)= estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
- **Step 4:** Expand the node n, and generate the successors of node n.
- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

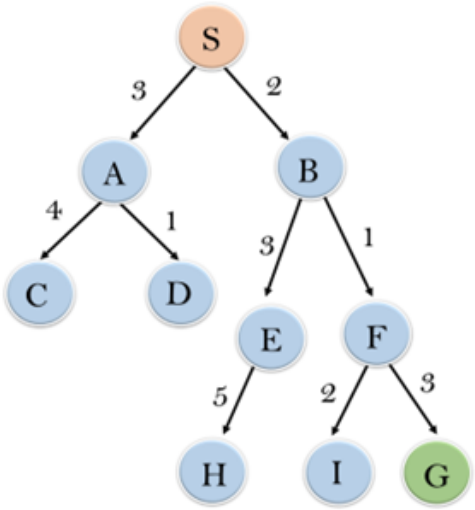
Disadvantages:

- It can behave as an unguided depth- first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

Example:

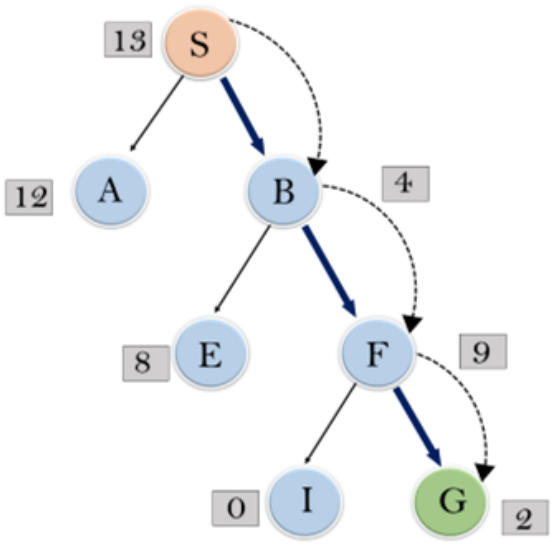
Consider the below search problem, and we will traverse it using greedy best-first

search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: **S- - - - > B- - - - - >F- - - - > G**

Time Complexity: The worst case time complexity of Greedy best first search is $O(b^m)$.

Space Complexity: The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

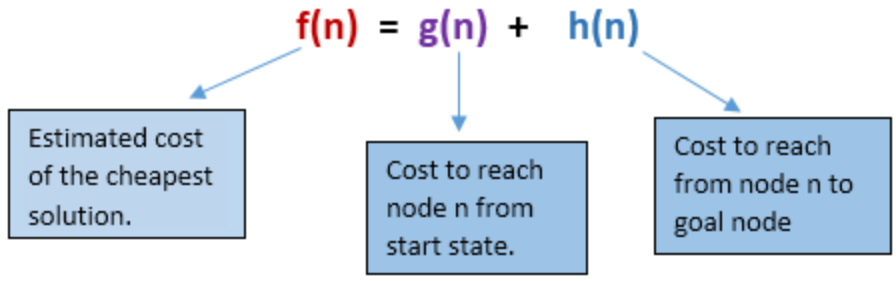
Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

2.) A* Search Algorithm:

A* search is the most commonly known form of best-first search. It uses heuristic function h(n), and cost to reach the node n from the start state g(n). It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number** .



At each point in the search space, only those node is expanded which have the lowest value of f(n), and the algorithm terminates when the goal node is found.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached

to the back pointer which reflects the lowest g(n') value.

Step 6: Return to **Step 2** .

Advantages:

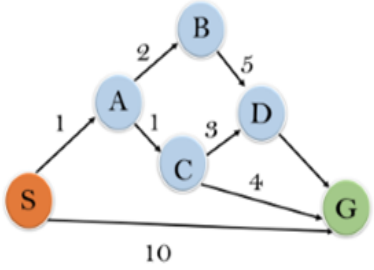
- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large- scale problems.

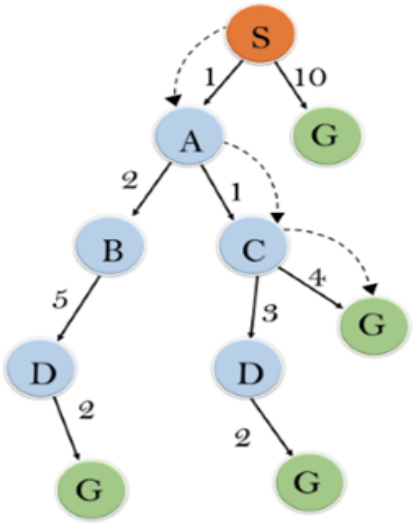
Example:

In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula f(n)= g(n) + h(n), where g(n) is the cost to reach any node from start state.
Here we will use OPEN and CLOSED list.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



Initialization: {(S, 5)}

Iteration1: {(S- - > A, 4), (S- - >G, 10)}

Iteration2: {(S- - > A- - >C, 4), (S- - > A- - >B, 7), (S- - >G, 10)}

Iteration3: {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S--
>G, 10)}

Iteration 4 will give the final result, as **S—>A—>C—>G** it provides the optimal path with cost 6.

Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition f(n)<="" li="">

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So the time complexity is O(b^d), where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is **O(b^d)**

Local Search Algorithms and Optimization Problem:

The informed and uninformed search expands the nodes systematically in two ways:
keeping different paths in the memory and
selecting the best suitable path,Which leads to a solution state required to reach the goal node. But beyond these **"classical search algorithms** ," we have some **"local search algorithms" where the path cost does not matters, and only focus on solution- state needed to reach the goal node.**

A local search algorithm completes its task by traversing on a single current node rather than multiple paths and following the neighbors of that node generally. Although local search algorithms are not systematic, still they have the following two advantages:
Local search algorithms use a very little or constant amount of memory as they operate only on a single path.
Most often, they find a reasonable solution in large or infinite state spaces where the classical or systematic algorithms do not work.

Does the local search algorithm work for a pure optimized problem?

Yes, the local search algorithm works for pure optimized problems. A pure optimization problem is one where all the nodes can give a solution. But the target is to find the best state out of all according to the **objective function** . Unfortunately, the pure optimization problem fails to find high-quality solutions to reach the goal state from the current state.

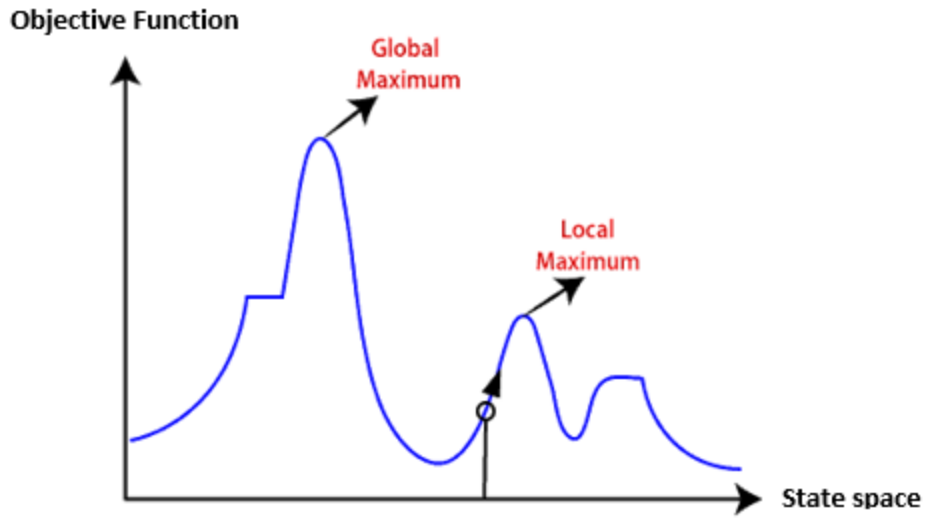
Note: An objective function is a function whose value is either minimized or maximized in different contexts of the optimization problems. In the case of search algorithms, an objective function can be the path cost for reaching the goal node, etc.

Working of a Local search algorithm

Let's understand the working of a local search algorithm with the help of an example:
Consider the below state- space landscape having both:

Location: It is defined by the state.

Elevation: It is defined by the value of the objective function or heuristic cost function.



A one-dimensional state-space landscape in which elevation corresponds to the objective function

The local search algorithm explores the above landscape by finding the following two points:

Global Minimum: If the elevation corresponds to the cost, then the task is to find the lowest valley, which is known as **Global Minimum.**

Global Maxima: If the elevation corresponds to an objective function, then it

finds the highest peak which is called as **Global Maxima** . It is the highest point in the valley.

We will understand the working of these points better in Hill- climbing search.

Below are some different types of local searches:

Hill- climbing Search
Simulated Annealing
Local Beam Search

Hill Climbing Algorithm in AI

Hill Climbing Algorithm : Hill climbing search is a local search problem. The purpose of the hill climbing search is to climb a hill and reach the topmost peak/ point of that hill. It is based on the **heuristic search technique** where the person who is climbing up on the hill estimates the direction which will lead him to the highest peak.

State- space Landscape of Hill climbing algorithm

To understand the concept of hill climbing algorithm, consider the below landscape representing the **goal state/peak** and the **current state** of the climber. The topographical regions shown in the figure can be defined as:

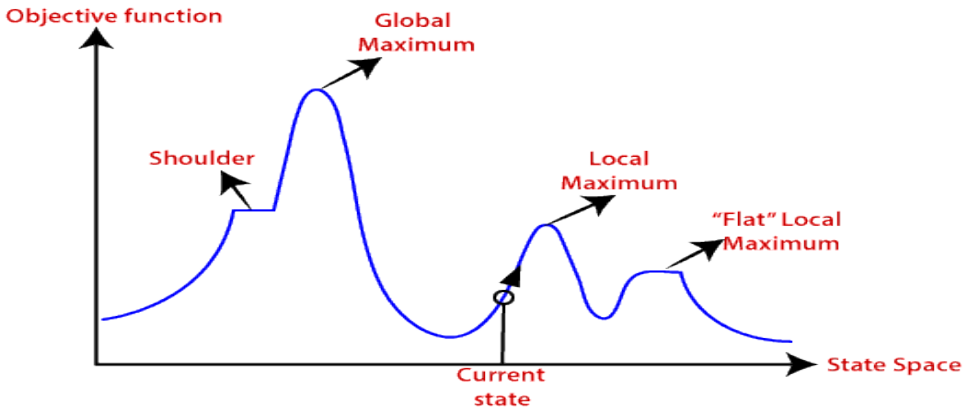
Global Maximum: It is the highest point on the hill, which is the goal state.

Local Maximum: It is the peak higher than all other peaks but lower than the global maximum.

Flat local maximum: It is the flat area over the hill where it has no uphill or downhill. It is a saturated point of the hill.

Shoulder: It is also a flat area where the summit is possible.

Current state: It is the current position of the person.

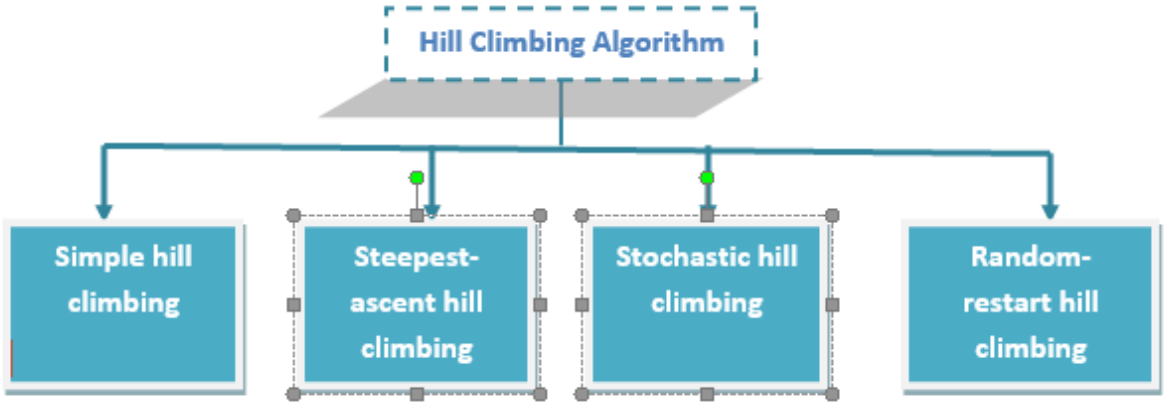


A one-dimensional state-space landscape in which elevation corresponds to the objective function

Types of Hill climbing search algorithm

There are following types of hill- climbing search:

- Simple hill climbing
- Steepest- ascent hill climbing
- Stochastic hill climbing
- Random- restart hill climbing



Simple hill climbing search

Simple hill climbing is the simplest technique to climb a hill. The task is to reach the highest peak of the mountain. Here, the movement of the climber depends on his move/steps. If he finds his next step better than the previous one, he continues to move else remain in the same state. This search focus only on his previous and next

step.

Simple hill climbing Algorithm

1. Create a **CURRENT** node, **NEIGHBOUR** node, and a **GOAL** node.
2. If the **CURRENT node=GOAL node** , return **GOAL** and terminate the search.
3. Else **CURRENT node<= NEIGHBOUR node,** move ahead.
4. Loop until the goal is not reached or a point is not found.

Steepest- ascent hill climbing

Steepest-ascent hill climbing is different from simple hill climbing search. Unlike simple hill climbing search, It considers all the successive nodes, compares them, and choose the node which is closest to the solution. Steepest hill climbing search is similar to **best-first search** because it focuses on each node instead of one.

Note: Both simple, as well as steepest-ascent hill climbing search, fails when there is no closer node.

Steepest- ascent hill climbing algorithm

1. Create a **CURRENT** node and a **GOAL** node.
2. If the **CURRENT node=GOAL** node, return **GOAL** and terminate the search.
3. Loop until a better node is not found to reach the solution.
4. If there is any better successor node present, expand it.
5. When the **GOAL** is attained, return **GOAL** and terminate.

Stochastic hill climbing

Stochastic hill climbing does not focus on all the nodes. It selects one node at random and decides whether it should be expanded or search for a better one.

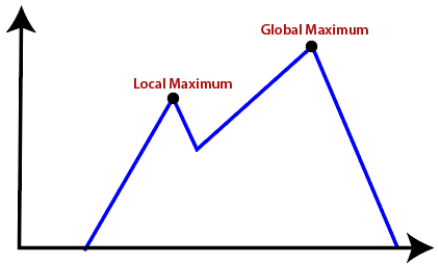
Random- restart hill climbing

Random-restart algorithm is based on **try and try strategy** . It iteratively searches the node and selects the best one at each step until the goal is not found. The success depends most commonly on the shape of the hill. If there are few plateaus, local maxima, and ridges, it becomes easy to reach the destination.

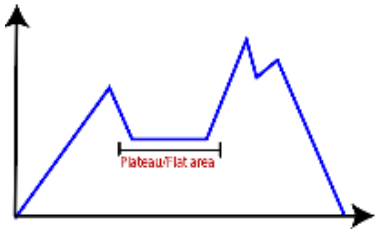
Limitations of Hill climbing algorithm

Hill climbing algorithm is a fast and furious approach. It finds the solution state rapidly because it is quite easy to improve a bad state. But, there are following limitations of this search:

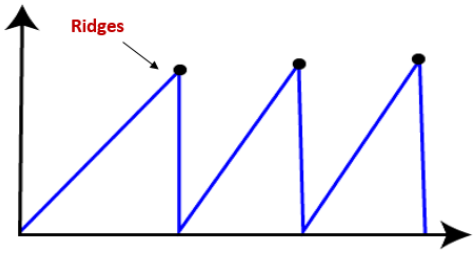
Local Maxima: It is that peak of the mountain which is highest than all its neighboring states but lower than the global maxima. It is not the goal peak because there is another peak higher than it.



Plateau: It is a flat surface area where no uphill exists. It becomes difficult for the climber to decide that in which direction he should move to reach the goal point. Sometimes, the person gets lost in the flat area.



Ridges: It is a challenging problem where the person finds two or more local maxima of the same height commonly. It becomes difficult for the person to navigate the right point and stuck to that point itself.



Simulated Annealing

Simulated annealing is similar to the hill climbing algorithm. It works on the current situation. It picks **a random move** instead of picking **the best move** . If the move leads to the improvement of the current situation, it is always accepted as a step towards the solution state, else it accepts the move having **a probability less than 1** . This search technique was first used in **1980** to solve **VLSI layout** problems. It is also applied for factory scheduling and other large optimization tasks.

Simulated Annealing in AI

It is used to find a **near-optimal solution** to difficult optimization problems, especially

when there are many possible solutions.

Basic idea:

Start with a random solution.
Make a small change to get a new solution.
If the new solution is better, accept it.
If it is worse, **sometimes accept it anyway** .
Gradually reduce a parameter called **temperature** , making the algorithm less willing to accept worse solutions.
Eventually, it settles on a good solution.

The probability of accepting a worse solution is commonly:

$$P=e^{-\Delta E/T}$$

where:

ΔE = how much worse the new solution is
T = current temperature
e = Euler's number

Why accept a worse solution?

This helps the algorithm **escape local optima** .

For example, imagine searching for the lowest point in a landscape. A normal hill-climbing algorithm might get stuck in a small valley. Simulated annealing can temporarily move uphill and then discover a deeper valley.

Simple algorithm

1. Choose an initial solution
2. Set a high temperature T
3. Repeat:
   a. Generate a neighboring solution
   b. Calculate the change in cost ΔE
   c. If the new solution is better:
      accept it
      Else:
      accept it with probability e^ (- ΔE/T)
   d. Reduce T
4. Return the best solution found

Applications

Traveling Salesman Problem (TSP)
Scheduling
Circuit design
Resource allocation
Feature selection
Neural- network/AI optimization

In one sentence: Simulated annealing is a probabilistic optimization technique that occasionally accepts worse solutions early on so it can escape local optima and eventually converge toward a good solution.

example

Sure! Here is a **simple example of Simulated Annealing in AI** .

Example: Finding the minimum value

Suppose we want to find the minimum of:
f(x)=x2
The best answer is obviously **x = 0** , but imagine the problem is much more complicated and we don't know where the minimum is.
Suppose we start at:
x=5
So:
f(5)=25
Now Simulated Annealing tries a neighboring value, say **x = 4** :
f(4)=16
Since 16 is better than 25, we **accept x = 4** .
Then it tries **x = 3** :
f(3)=9
Again, better → **accept** .
Now imagine we're at a point where the next move makes the solution worse.
Simulated Annealing may **still accept the worse move with some probability** .
For example:

Current x	**New x**	**Current cost**	**New cost**	**Decision**
5	4	25	16	Accept
4	3	16	9	Accept
3	4	9	16	Maybe accept
4	2	16	4	Accept
2	1	4	1	Accept
1	0	1	0	Accept

The important part is the **"maybe accept"** step. Early in the process, the temperature is high, so the algorithm is more willing to accept a worse solution. As the temperature decreases, it becomes increasingly selective.

Local Beam Search

Local beam search is quite different from random-restart search. It keeps track of **k** states instead of just one. It selects **k** randomly generated states, and expand them at each step. If any state is a goal state, the search stops with success. Else it selects the best **k** successors from the complete list and repeats the same process. In random-restart search where each search process runs independently, but in local beam search, the necessary information is shared between the parallel search processes.

Local Beam Search in AI

Local Beam Search is a search algorithm that keeps **multiple states**

(solutions) at the same time , instead of following only one state.

Simple idea

Suppose we want to find the best route or solve a puzzle.

Instead of:

Start with **1 solution** → improve it → improve it → goal

Local Beam Search does:

Start with **k solutions** → generate their successors → keep the **best k** → repeat

Here, **k** is the number of states we keep.

Example

Let **k = 3** .

We start with 3 states:

A, B, C

Their possible successors are:

A → D, E, F

B → G, H, I

C → J, K, L

Now we have **9 possible states** :

D, E, F, G, H, I, J, K, L

Suppose their evaluation scores are:

State	Score
D	8
E	5
F	9
G	3
H	7
I	4
J	6
K	2
L	10

If we want the **3 best states** , we keep:

L (10), F (9), D (8)

The other states are discarded.

Then we generate successors of **L, F, and D** , evaluate them, and again keep only the best 3.

This continues until a goal state is found.

Difference from Hill Climbing

Hill Climbing	**Local Beam Search**
Keeps **1 state**	Keeps **k states**
Searches one path	Searches multiple paths
Can easily get stuck in a local optimum	More likely to escape local problems
Simple	More computationally expensive

Disadvantages of Local Beam search

This search can suffer from a lack of diversity among the **k** states.

It is an expensive version of hill climbing search.

Local Search in Continuous Spaces

Local search in continuous spaces is used when the possible solutions are **real-valued variables** rather than discrete states.

For example:

x=2.5,y=7.3

Instead of moving from one fixed state to another, the algorithm can make **small changes** to the values.

Example

Suppose we want to minimize:

f(x)=(x−5)2

The minimum occurs at:

x=5

Start with:

x=2

Then calculate:

f(2)=(2−5)2=9

The algorithm tries nearby values:

x	**f(x)**
2.0	9.00
2	6.

.5	25
3.5	2.25
4.0	1.00
4.5	0.25
5.0	**0.00**

It gradually moves toward **x = 5** , where the function has its minimum.

Common methods

For continuous spaces, local search can use:

Gradient descent – moves in the direction that reduces the function.

Simulated annealing – sometimes accepts worse solutions to escape local optima.

Evolutionary algorithms – maintains and improves a population of solutions.

Random-restart search – starts from different random points to avoid getting stuck.

Simple example: Gradient Descent

For:

$f(x)=x2$

The derivative is:

$f'(x)=2x$

The update rule is:

$x_{new} = x - \alpha(2x)$

where α is the **learning rate** .

If:

$x=10, \alpha=0.1$

then:

$x_{new} = 10 - 0.1(20) = 8$

Next:

$8 \rightarrow 6.4 \rightarrow 5.12 \rightarrow 4.096 \rightarrow \quad \rightarrow 0$

So the algorithm gradually approaches the optimal value **x = 0** .

In short: Local search in continuous spaces searches for the best solution by making **small changes to continuous variables** , such as real numbers, rather than moving between discrete states.

Searching with Non- Deterministic Actions in AI

In AI, non-deterministic actions are actions where the result is not completely predictable.

An agent chooses an action, but several different outcomes are possible.

Simple Example

Suppose a robot wants to move from A to B.

It chooses:

Move Forward

But because the environment is uncertain, the robot may:

Move Forward

|

↓ ↓ ↓

B A C

Success Stays Wrong way

The agent must be prepared for all possible outcomes.

Example: Vacuum Cleaner

A vacuum cleaner is in Room A and wants to reach Room B.

Searching with Partial Observations in AI

Searching with partial observations means the AI **cannot see the complete state of the environment** . It only receives incomplete or uncertain information about the current situation.

Simple example

Imagine a robot moving through rooms, but it cannot see inside every room.

The robot knows:

"I am somewhere in Room A or Room B, but I don't know exactly where."

This is a **partial observation** .

Robot

|

↓ ↓

Room A Room B

Possible Possible

The AI must make decisions even though it doesn't know its exact state.

Example: Robot navigation

Suppose a robot has three possible locations:

A, B, C

But its sensors only tell it:

"There is a wall in front."

The robot cannot determine exactly which room it is in.

So its **belief state** is:

Belief={A,B,C}

After taking an action, the robot receives a new observation and updates its belief.

For example:

Initial belief:

{A, B, C}

↓ Move Forward

New observation:

"Door detected"

↓

Updated belief:

{B, C}

The AI continues searching using this **set of possible states** .

Belief State

A **belief state** represents all the states that the AI thinks are currently possible.

For example:

B={S1,S2,S3}

means:

"The real state could be S_1, S_2, or S_3."

The AI searches over **belief states** , rather than individual physical states.

Online search Agents and Unknown Environments:

An online search agent is an AI agent that searches for a solution while interacting with the environment. It does not know the complete environment beforehand.

Simple Example

Imagine a robot that needs to reach a destination in an unknown building.

The robot doesn't have a map:

Start

↓

Choose direction

↓

Move

↓

Observe environment

↓

Learn new information

↓

Choose next direction

↓

Repeat

↓

Goal

The robot discovers the environment as it moves.

Example

Suppose the robot starts at A:

A → B → ?

After moving from A to B, it discovers that B has two possible paths:



The robot tries C, discovers it is a dead end, and then returns to B:

A → B → C → B → D → E → G